```
            a=c;
        }
        u *= c;
        sn=sin(u);
        cn=cos(u);
        if (sn != 0.0) {
            a=cn/sn;
            c *= a;
            for (ii=l;ii>=0;ii--) {
                b=em[ii];
                a *= c;
                c *= dn;
                dn=(en[ii]+a)/(b+a);
                a=c/b;
            }
            a=1.0/sqrt(c*c+1.0);
            sn=(sn >= 0.0 ? a : -a);
            cn=c*sn;
        }
        if (bo) {
            a=dn;
            dn=cn;
            cn=a;
            sn /= d;
        }
    } else {
        cn=1.0/cosh(u);
        dn=cn;
        sn=tanh(u);
    }
}
```

CITED REFERENCES AND FURTHER READING:

Erdélyi, A., Magnus, W., Oberhettinger, F., and Tricomi, F.G. 1953, *Higher Transcendental Functions*, Vol. II, (New York: McGraw-Hill).[1]

Gradshteyn, I.S., and Ryzhik, I.W. 1980, *Table of Integrals, Series, and Products* (New York: Academic Press).[2]

Carlson, B.C. 1977, "Elliptic Integrals of the First Kind," *SIAM Journal on Mathematical Analysis*, vol. 8, pp. 231–242.[3]

Carlson, B.C. 1987, "A Table of Elliptic Integrals of the Second Kind," *Mathematics of Computation*, vol. 49, pp. 595–606[4]; 1988, "A Table of Elliptic Integrals of the Third Kind," *op. cit.*, vol. 51, pp. 267–280[5]; 1989, "A Table of Elliptic Integrals: Cubic Cases," *op. cit.*, vol. 53, pp. 327–333[6]; 1991, "A Table of Elliptic Integrals: One Quadratic Factor," *op. cit.*, vol. 56, pp. 267–280.[7]

Carlson, B.C., and FitzSimons, J. 2000, "Reduction Theorems for Elliptic Integrands with the Square Root of Two Quadratic Factors," *Journal of Computational and Applied Mathematics*, vol. 118, pp. 71–85.[8]

Bulirsch, R. 1965, "Numerical Calculation of Elliptic Integrals and Elliptic Functions," *Numerische Mathematik*, vol. 7, pp. 78–90; 1965, *op. cit.*, vol. 7, pp. 353–354; 1969, *op. cit.*, vol. 13, pp. 305–315.[9]

Carlson, B.C. 1979, "Computing Elliptic Integrals by Duplication," *Numerische Mathematik*, vol. 33, pp. 1–16.[10]

Carlson, B.C., and Notis, E.M. 1981, "Algorithms for Incomplete Elliptic Integrals," *ACM Transactions on Mathematical Software*, vol. 7, pp. 398–403.[11]

Carlson, B.C. 1978, "Short Proofs of Three Theorems on Elliptic Integrals," *SIAM Journal on Mathematical Analysis*, vol. 9, p. 524–528.[12]

Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at `http://www.nr.com/aands`, Chapter 17.[13]

Mathews, J., and Walker, R.L. 1970, *Mathematical Methods of Physics*, 2nd ed. (Reading, MA: W.A. Benjamin/Addison-Wesley), pp. 78–79.

6.13 Hypergeometric Functions

As was discussed in §5.14, a fast, general routine for the the complex hypergeometric function ${}_2F_1(a,b,c;z)$ is difficult or impossible. The function is defined as the analytic continuation of the hypergeometric series

$$
\begin{aligned}
{}_2F_1(a,b,c;z) = 1 + \frac{ab}{c}\frac{z}{1!} + \frac{a(a+1)b(b+1)}{c(c+1)}\frac{z^2}{2!} + \cdots \\
+ \frac{a(a+1)\ldots(a+j-1)b(b+1)\ldots(b+j-1)}{c(c+1)\ldots(c+j-1)}\frac{z^j}{j!} + \cdots
\end{aligned}
\tag{6.13.1}
$$

This series converges only within the unit circle $|z| < 1$ (see [1]), but one's interest in the function is not confined to this region.

Section 5.14 discussed the method of evaluating this function by direct path integration in the complex plane. We here merely list the routines that result.

Implementation of the function `hypgeo` is straightforward and is described by comments in the program. The machinery associated with Chapter 17's routine for integrating differential equations, `Odeint`, is only minimally intrusive and need not even be completely understood: Use of `Odeint` requires one function call to the constructor, with a prescribed format for the derivative routine `Hypderiv`, followed by a call to the `integrate` method.

The function `hypgeo` will fail, of course, for values of z too close to the singularity at 1. (If you need to approach this singularity, or the one at ∞, use the "linear transformation formulas" in §15.3 of [1].) Away from $z = 1$, and for moderate values of a, b, c, it is often remarkable how few steps are required to integrate the equations. A half-dozen is typical.

hypgeo.h

```
Complex hypgeo(const Complex &a, const Complex &b,const Complex &c,
    const Complex &z)
```

Complex hypergeometric function ${}_2F_1$ for complex a, b, c, and z, by direct integration of the hypergeometric equation in the complex plane. The branch cut is taken to lie along the real axis, Re $z > 1$.

```
{
    const Doub atol=1.0e-14,rtol=1.0e-14;      Accuracy parameters.
    Complex ans,dz,z0,y[2];
    VecDoub yy(4);
    if (norm(z) <= 0.25) {                     Use series...
        hypser(a,b,c,z,ans,y[1]);
        return ans;
    }
    ...or pick a starting point for the path integration.
    else if (real(z) < 0.0) z0=Complex(-0.5,0.0);
    else if (real(z) <= 1.0) z0=Complex(0.5,0.0);
```

```
    else z0=Complex(0.0,imag(z) >= 0.0 ? 0.5 : -0.5);
    dz=z-z0;
    hypser(a,b,c,z0,y[0],y[1]);                Get starting function and derivative.
    yy[0]=real(y[0]);
    yy[1]=imag(y[0]);
    yy[2]=real(y[1]);
    yy[3]=imag(y[1]);
    Hypderiv d(a,b,c,z0,dz);                   Set up the functor for the derivatives.
    Output out;                                Suppress output in Odeint.
    Odeint<StepperBS<Hypderiv> > ode(yy,0.0,1.0,atol,rtol,0.1,0.0,out,d);
```

The arguments to `Odeint` are the vector of independent variables, the starting and ending values of the dependent variable, the accuracy parameters, an initial guess for the stepsize, a minimum stepsize, and the names of the output object and the derivative object. The integration is performed by the Bulirsch-Stoer stepping routine.

```
    ode.integrate();
    y[0]=Complex(yy[0],yy[1]);
    return y[0];
}
```

```
void hypser(const Complex &a, const Complex &b, const Complex &c,                    hypgeo.h
    const Complex &z, Complex &series, Complex &deriv)
```

Returns the hypergeometric series ${}_2F_1$ and its derivative, iterating to machine accuracy. For $|z| \leq 1/2$ convergence is quite rapid.

```
{
    deriv=0.0;
    Complex fac=1.0;
    Complex temp=fac;
    Complex aa=a;
    Complex bb=b;
    Complex cc=c;
    for (Int n=1;n<=1000;n++) {
        fac *= ((aa*bb)/cc);
        deriv += fac;
        fac *= ((1.0/n)*z);
        series=temp+fac;
        if (series == temp) return;
        temp=series;
        aa += 1.0;
        bb += 1.0;
        cc += 1.0;
    }
    throw("convergence failure in hypser");
}
```

```
struct Hypderiv {                                                                    hypgeo.h
```

Functor to compute derivatives for the hypergeometric equation; see text equation (5.14.4).

```
    Complex a,b,c,z0,dz;
    Hypderiv(const Complex &aa, const Complex &bb,
        const Complex &cc, const Complex &z00,
        const Complex &dzz) : a(aa),b(bb),c(cc),z0(z00),dz(dzz) {}
    void operator() (const Doub s, VecDoub_I &yy, VecDoub_O &dyyds) {
        Complex z,y[2],dyds[2];
        y[0]=Complex(yy[0],yy[1]);
        y[1]=Complex(yy[2],yy[3]);
        z=z0+s*dz;
        dyds[0]=y[1]*dz;
        dyds[1]=(a*b*y[0]-(c-(a+b+1.0)*z)*y[1])*dz/(z*(1.0-z));
        dyyds[0]=real(dyds[0]);
        dyyds[1]=imag(dyds[0]);
        dyyds[2]=real(dyds[1]);
        dyyds[3]=imag(dyds[1]);
    }
};
```

CITED REFERENCES AND FURTHER READING:

Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at `http://www.nr.com/aands`.[1]

6.14 Statistical Functions

Certain special functions get frequent use because of their relation to common univariate statistical distributions, that is, probability densities in a single variable. In this section we survey a number of such common distributions in a unified way, giving, in each case, routines for computing the probability density function $p(x)$; the cumulative density function or *cdf*, written $P(< x)$; and the inverse of the cumulative density function $x(P)$. The latter function is needed for finding the values of x associated with specified *percentile points* or *quantiles* in significance tests, for example, the 0.5%, 5%, 95% or 99.5% points.

The emphasis of this section is on defining and computing these statistical functions. Section §7.3 is a related section that discusses how to generate random deviates from the distributions discussed here. We defer discussion of the actual use of these distributions in statistical tests to Chapter 14.

6.14.1 Normal (Gaussian) Distribution

If x is drawn from a *normal distribution* with mean μ and standard deviation σ, then we write

$$\begin{aligned} x &\sim \mathrm{N}(\mu,\sigma), \qquad \sigma > 0 \\ p(x) &= \frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{1}{2}\left[\frac{x-\mu}{\sigma}\right]^2\right) \end{aligned} \tag{6.14.1}$$

with $p(x)$ the probability density function. Note the special use of the notation "$\sim$" in this section, which can be read as "is drawn from a distribution." The variance of the distribution is, of course, σ^2.

The cumulative distribution function is the probability of a value $\le x$. For the normal distribution, this is given in terms of the complementary error function by

$$\mathrm{cdf} \equiv P(< x) \equiv \int_{-\infty}^{x} p(x')dx' = \frac{1}{2}\mathrm{erfc}\left(-\frac{1}{\sqrt{2}}\left[\frac{x-\mu}{\sigma}\right]\right) \tag{6.14.2}$$

The inverse cdf can thus be calculated in terms of the inverse of erfc,

$$x(P) = \mu - \sqrt{2}\sigma\,\mathrm{erfc}^{-1}(2P) \tag{6.14.3}$$

The following structure implements the above relations.

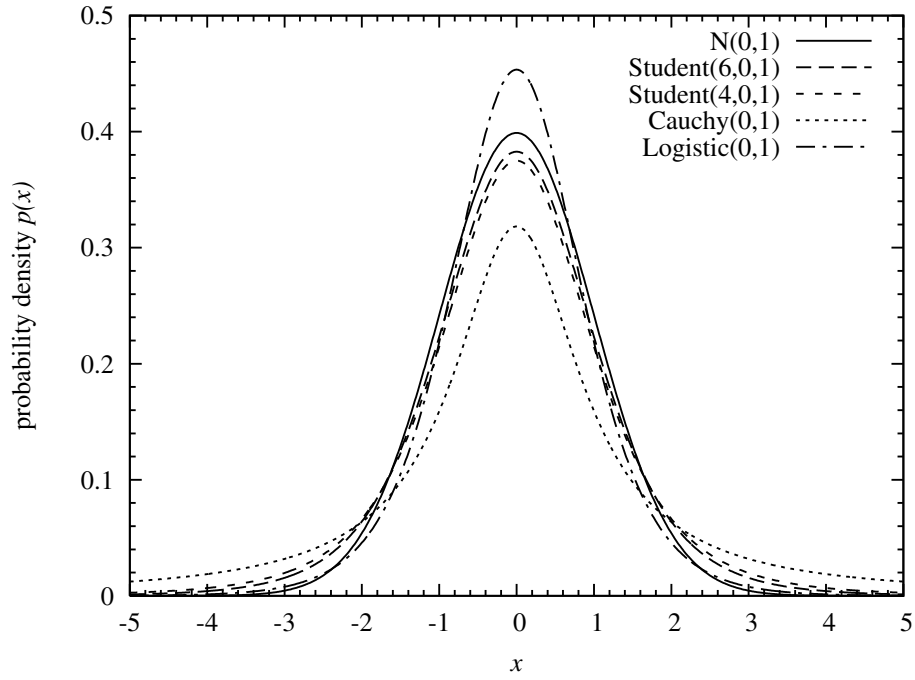

Figure 6.14.1. Examples of centrally peaked distributions that are symmetric on the real line. Any of these can substitute for the normal distribution either as an approximation or in applications such as robust estimation. They differ largely in the decay rate of their tails.

erf.h

```
struct Normaldist : Erf {
Normal distribution, derived from the error function Erf.
    Doub mu, sig;
    Normaldist(Doub mmu = 0., Doub ssig = 1.) : mu(mmu), sig(ssig) {
    Constructor. Initialize with μ and σ. The default with no arguments is N(0, 1).
        if (sig <= 0.) throw("bad sig in Normaldist");
    }
    Doub p(Doub x) {
    Return probability density function.
        return (0.398942280401432678/sig)*exp(-0.5*SQR((x-mu)/sig));
    }
    Doub cdf(Doub x) {
    Return cumulative distribution function.
        return 0.5*erfc(-0.707106781186547524*(x-mu)/sig);
    }
    Doub invcdf(Doub p) {
    Return inverse cumulative distribution function.
        if (p <= 0. || p >= 1.) throw("bad p in Normaldist");
        return -1.41421356237309505*sig*inverfc(2.*p)+mu;
    }
};
```

We will use the conventions of the above code for all the distributions in this section. A distribution's parameters (here, μ and σ) are set by the constructor and then referenced as needed by the member functions. The density function is always `p()`, the cdf is `cdf()`, and the inverse cdf is `invcdf()`. We will generally check the arguments of probability functions for validity, since many program bugs can show up as, e.g., a probability out of the range $[0, 1]$.

6.14.2 Cauchy Distribution

Like the normal distribution, the *Cauchy distribution* is a centrally peaked, symmetric distribution with a parameter μ that specifies its center and a parameter σ that specifies its width. *Unlike* the normal distribution, the Cauchy distribution has tails that decay very slowly at infinity, as $|x|^{-2}$, so slowly that moments higher than the zeroth moment (the area under the curve) don't even exist. The parameter μ is therefore, strictly speaking, not the mean, and the parameter σ is not, technically, the standard deviation. But these two parameters substitute for those moments as measures of central position and width.

The defining probability density is

$$x \sim \text{Cauchy}(\mu, \sigma), \qquad \sigma > 0$$
$$p(x) = \frac{1}{\pi\sigma}\left(1 + \left[\frac{x-\mu}{\sigma}\right]^2\right)^{-1} \tag{6.14.4}$$

If $x \sim \text{Cauchy}(0,1)$, then also $1/x \sim \text{Cauchy}(0,1)$ and also $(ax+b)^{-1} \sim \text{Cauchy}(-b/a, 1/a)$.

The cdf is given by

$$\text{cdf} \equiv P(<x) \equiv \int_{-\infty}^{x} p(x')dx' = \frac{1}{2} + \frac{1}{\pi}\arctan\left(\frac{x-\mu}{\sigma}\right) \tag{6.14.5}$$

The inverse cdf is given by

$$x(P) = \mu + \sigma\tan\left(\pi[P - \tfrac{1}{2}]\right) \tag{6.14.6}$$

Figure 6.14.1 shows Cauchy$(0,1)$ as compared to the normal distribution N$(0,1)$, as well as several other similarly shaped distributions discussed below.

The Cauchy distribution is sometimes called the *Lorentzian distribution*.

distributions.h

```
struct Cauchydist {
```
Cauchy distribution.
```
    Doub mu, sig;
    Cauchydist(Doub mmu = 0., Doub ssig = 1.) : mu(mmu), sig(ssig) {
```
    Constructor. Initialize with μ and σ. The default with no arguments is Cauchy$(0,1)$.
```
        if (sig <= 0.) throw("bad sig in Cauchydist");
    }
    Doub p(Doub x) {
```
    Return probability density function.
```
        return 0.318309886183790671/(sig*(1.+SQR((x-mu)/sig)));
    }
    Doub cdf(Doub x) {
```
    Return cumulative distribution function.
```
        return 0.5+0.318309886183790671*atan2(x-mu,sig);
    }
    Doub invcdf(Doub p) {
```
    Return inverse cumulative distribution function.
```
        if (p <= 0. || p >= 1.) throw("bad p in Cauchydist");
        return mu + sig*tan(3.14159265358979324*(p-0.5));
    }
};
```

6.14.3 Student-t Distribution

A generalization of the Cauchy distribution is the Student-t distribution, named for the early 20th century statistician William Gosset, who published under the name "Student" because his employer, Guinness Breweries, required him to use a pseudonym. Like the Cauchy distribution, the Student-t distribution has power-law tails at infinity, but it has an additional parameter ν that specifies how rapidly they decay, namely as $|t|^{-(\nu+1)}$. When ν is an integer, the number of convergent moments, including the zeroth, is thus ν.

The defining probability density (conventionally written in a variable t instead of x) is

$$t \sim \text{Student}(\nu, \mu, \sigma), \qquad \nu > 0,\ \sigma > 0$$
$$p(t) = \frac{\Gamma(\frac{1}{2}[\nu+1])}{\Gamma(\frac{1}{2}\nu)\sqrt{\nu\pi}\sigma}\left(1 + \frac{1}{\nu}\left[\frac{t-\mu}{\sigma}\right]^2\right)^{-\frac{1}{2}(\nu+1)} \tag{6.14.7}$$

The Cauchy distribution is obtained in the case $\nu = 1$. In the opposite limit, $\nu \to \infty$, the normal distribution is obtained. In pre-computer days, this was the basis of various approximation schemes for the normal distribution, now all generally irrelevant. Figure 6.14.1 shows examples of the Student-t distribution for $\nu = 1$ (Cauchy), $\nu = 4$, and $\nu = 6$. The approach to the normal distribution is evident.

The mean of Student(ν, μ, σ) is (by symmetry) μ. The variance is not σ^2, but rather

$$\text{Var}\{\text{Student}(\nu, \mu, \sigma)\} = \frac{\nu}{\nu - 2}\sigma^2 \tag{6.14.8}$$

For additional moments, and other properties, see [1].

The cdf is given by an incomplete beta function. If we let

$$x \equiv \frac{\nu}{\nu + \left(\frac{t-\mu}{\sigma}\right)^2} \tag{6.14.9}$$

then

$$\text{cdf} \equiv P(<t) \equiv \int_{-\infty}^{t} p(t')dt' = \begin{cases} \frac{1}{2}I_x(\frac{1}{2}\nu, \frac{1}{2}), & t \le \mu \\ 1 - \frac{1}{2}I_x(\frac{1}{2}\nu, \frac{1}{2}), & t > \mu \end{cases} \tag{6.14.10}$$

The inverse cdf is given by an inverse incomplete beta function (see code below for the exact formulation).

In practice, the Student-t cdf is the above form is rarely used, since most statistical tests using Student-t are double-sided. Conventionally, the two-tailed function $A(t|\nu)$ is defined (only) for the case $\mu = 0$ and $\sigma = 1$ by

$$A(t|\nu) \equiv \int_{-t}^{+t} p(t')dt' = 1 - I_x(\tfrac{1}{2}\nu, \tfrac{1}{2}) \tag{6.14.11}$$

with x as given above. The statistic $A(t|\nu)$ is notably used in the test of whether two observed distributions have the same mean. The code below implements both equations (6.14.10) and (6.14.11), as well as their inverses.

ncgammabeta.h

```
struct Studenttdist : Beta {
```
Student-t distribution, derived from the beta function Beta.
```
    Doub nu, mu, sig, np, fac;
    Studenttdist(Doub nnu, Doub mmu = 0., Doub ssig = 1.)
    : nu(nnu), mu(mmu), sig(ssig) {
```
Constructor. Initialize with ν, μ and σ. The default with one argument is Student$(\nu, 0, 1)$.
```
        if (sig <= 0. || nu <= 0.) throw("bad sig,nu in Studentdist");
        np = 0.5*(nu + 1.);
        fac = gammln(np)-gammln(0.5*nu);
    }
    Doub p(Doub t) {
```
Return probability density function.
```
        return exp(-np*log(1.+SQR((t-mu)/sig)/nu)+fac)
            /(sqrt(3.14159265358979324*nu)*sig);
    }
    Doub cdf(Doub t) {
```
Return cumulative distribution function.
```
        Doub p = 0.5*betai(0.5*nu, 0.5, nu/(nu+SQR((t-mu)/sig)));
        if (t >= mu) return 1. - p;
        else return p;
    }
    Doub invcdf(Doub p) {
```
Return inverse cumulative distribution function.
```
        if (p <= 0. || p >= 1.) throw("bad p in Studentdist");
        Doub x = invbetai(2.*MIN(p,1.-p), 0.5*nu, 0.5);
        x = sig*sqrt(nu*(1.-x)/x);
        return (p >= 0.5? mu+x : mu-x);
    }
    Doub aa(Doub t) {
```
Return the two-tailed cdf $A(t|\nu)$.
```
        if (t < 0.) throw("bad t in Studentdist");
        return 1.-betai(0.5*nu, 0.5, nu/(nu+SQR(t)));
    }
    Doub invaa(Doub p) {
```
Return the inverse, namely t such that $p = A(t|\nu)$.
```
        if (p < 0. || p >= 1.) throw("bad p in Studentdist");
        Doub x = invbetai(1.-p, 0.5*nu, 0.5);
        return sqrt(nu*(1.-x)/x);
    }
};
```

6.14.4 Logistic Distribution

The *logistic distribution* is another symmetric, centrally peaked distribution that can be used instead of the normal distribution. Its tails decay exponentially, but still much more slowly than the normal distribution's "exponent of the square."

The defining probability density is

$$p(y) = \frac{e^{-y}}{(1+e^{-y})^2} = \frac{e^{y}}{(1+e^{y})^2} = \tfrac{1}{4}\operatorname{sech}^2\left(\tfrac{1}{2}y\right) \tag{6.14.12}$$

The three forms are algebraically equivalent, but, to avoid overflows, it is wise to use the negative and positive exponential forms for positive and negative values of y, respectively.

The variance of the distribution (6.14.12) turns out to be $\pi^2/3$. Since it is convenient to have parameters μ and σ with the conventional meanings of mean and standard deviation, equation (6.14.12) is often replaced by the *standardized logistic*

distribution,

$$x \sim \text{Logistic}(\mu, \sigma), \qquad \sigma > 0$$
$$p(x) = \frac{\pi}{4\sqrt{3}\sigma} \text{sech}^2 \left(\frac{\pi}{2\sqrt{3}} \left[\frac{x-\mu}{\sigma} \right] \right) \tag{6.14.13}$$

which implies equivalent forms using the positive and negative exponentials (see code below).

The cdf is given by

$$\text{cdf} \equiv P(< x) \equiv \int_{-\infty}^{x} p(x')dx' = \left[1 + \exp\left(-\frac{\pi}{\sqrt{3}} \left[\frac{x-\mu}{\sigma} \right] \right) \right]^{-1} \tag{6.14.14}$$

The inverse cdf is given by

$$x(P) = \mu + \frac{\sqrt{3}}{\pi} \sigma \log \left(\frac{P}{1-P} \right) \tag{6.14.15}$$

distributions.h

```
struct Logisticdist {
Logistic distribution.
    Doub mu, sig;
    Logisticdist(Doub mmu = 0., Doub ssig = 1.) : mu(mmu), sig(ssig) {
    Constructor. Initialize with μ and σ. The default with no arguments is Logistic(0, 1).
        if (sig <= 0.) throw("bad sig in Logisticdist");
    }
    Doub p(Doub x) {
    Return probability density function.
        Doub e = exp(-abs(1.81379936423421785*(x-mu)/sig));
        return 1.81379936423421785*e/(sig*SQR(1.+e));
    }
    Doub cdf(Doub x) {
    Return cumulative distribution function.
        Doub e = exp(-abs(1.81379936423421785*(x-mu)/sig));
        if (x >= mu) return 1./(1.+e);          Because we used abs to control over-
        else return e/(1.+e);                       flow, we now have two cases.
    }
    Doub invcdf(Doub p) {
    Return inverse cumulative distribution function.
        if (p <= 0. || p >= 1.) throw("bad p in Logisticdist");
        return mu + 0.5513288954217920 49*sig*log(p/(1.-p));
    }
};
```

The logistic distribution is cousin to the *logit transformation* that maps the open unit interval $0 < p < 1$ onto the real line $-\infty < u < \infty$ by the relation

$$u = \log \left(\frac{p}{1-p} \right) \tag{6.14.16}$$

Back when a book of tables and a slide rule were a statistician's working tools, the logit transformation was used to approximate processes on the interval by analytically simpler processes on the real line. A uniform distribution on the interval maps by the logit transformation to a logistic distribution on the real line. With the computer's ability to calculate distributions on the interval directly (beta distributions, for example), that motivation has vanished.

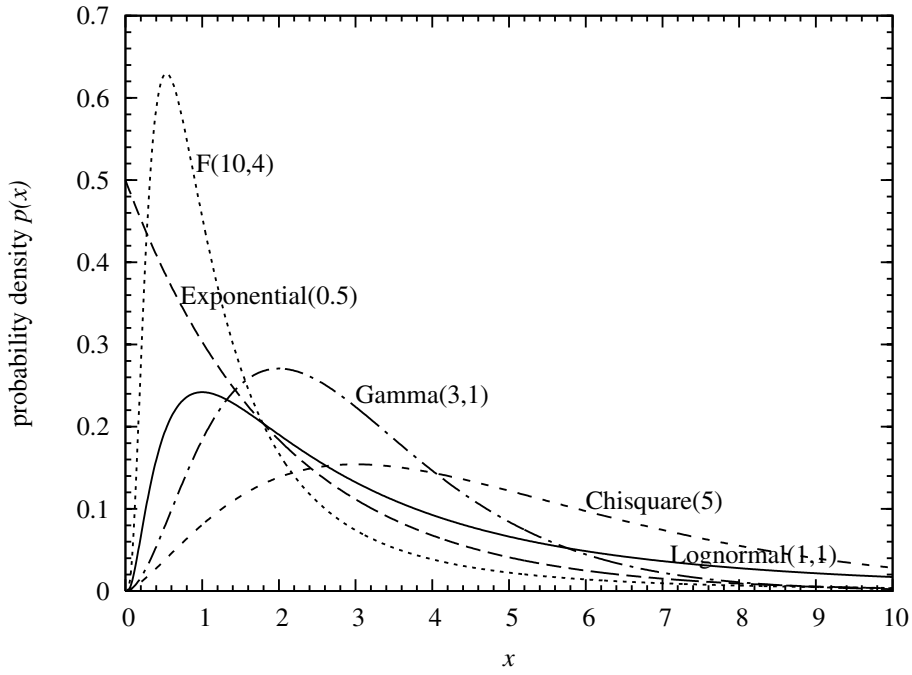

Figure 6.14.2. Examples of common distributions on the half-line $x > 0$.

Another cousin is the *logistic equation*,

$$\frac{dy}{dt} \propto y(y_{\max} - y) \tag{6.14.17}$$

a differential equation describing the growth of some quantity y, starting off as an exponential but reaching, asymptotically, a value $y_{\max}$. The solution of this equation is identical, up to a scaling, to the cdf of the logistic distribution.

6.14.5 Exponential Distribution

With the *exponential distribution* we now turn to common distribution functions defined on the positive real axis $x \geq 0$. Figure 6.14.2 shows examples of several of the distributions that we will discuss. The exponential is the simplest of them all. It has a parameter β that can control its width (in inverse relationship), but its mode is always at zero:

$$\begin{aligned} x &\sim \text{Exponential}(\beta), \qquad \beta > 0 \\ p(x) &= \beta \exp(-\beta x), \qquad x > 0 \end{aligned} \tag{6.14.18}$$

$$\text{cdf} \equiv P(< x) \equiv \int_0^x p(x')dx' = 1 - \exp(-\beta x) \tag{6.14.19}$$

$$x(P) = -\frac{1}{\beta}\log(1 - P) \tag{6.14.20}$$

The mean and standard deviation of the exponential distribution are both $1/\beta$. The median is $\log(2)/\beta$. Reference [1] has more to say about the exponential distribution than you would ever think possible.

distributions.h

```
struct Expondist {
Exponential distribution.
    Doub bet;
    Expondist(Doub bbet) : bet(bbet) {
    Constructor. Initialize with β.
        if (bet <= 0.) throw("bad bet in Expondist");
    }
    Doub p(Doub x) {
    Return probability density function.
        if (x < 0.) throw("bad x in Expondist");
        return bet*exp(-bet*x);
    }
    Doub cdf(Doub x) {
    Return cumulative distribution function.
        if (x < 0.) throw("bad x in Expondist");
        return 1.-exp(-bet*x);
    }
    Doub invcdf(Doub p) {
    Return inverse cumulative distribution function.
        if (p < 0. || p >= 1.) throw("bad p in Expondist");
        return -log(1.-p)/bet;
    }
};
```

6.14.6 Weibull Distribution

The Weibull distribution generalizes the exponential distribution in a way that is often useful in hazard, survival, or reliability studies. When the lifetime (time to failure) of an item is exponentially distributed, there is a constant probability per unit time that an item will fail, if it has not already done so. That is,

$$\text{hazard} \equiv \frac{p(x)}{P(>x)} \propto \text{constant} \tag{6.14.21}$$

Exponentially lived items don't age; they just keep rolling the same dice until, one day, their number comes up. In many other situations, however, it is observed that an item's hazard (as defined above) does change with time, say as a power law,

$$\frac{p(x)}{P(>x)} \propto x^{\alpha-1}, \qquad \alpha > 0 \tag{6.14.22}$$

The distribution that results is the Weibull distribution, named for Swedish physicist Waloddi Weibull, who used it as early as 1939. When $\alpha > 1$, the hazard increases with time, as for components that wear out. When $0 < \alpha < 1$, the hazard decreases with time, as for components that experience "infant mortality."

We say that

$$x \sim \text{Weibull}(\alpha, \beta) \qquad \text{iff} \qquad y \equiv \left(\frac{x}{\beta}\right)^{\alpha} \sim \text{Exponential}(1) \tag{6.14.23}$$

The probability density is

$$p(x) = \left(\frac{\alpha}{\beta}\right)\left(\frac{x}{\beta}\right)^{\alpha-1} e^{-(x/\beta)^{\alpha}}, \qquad x > 0 \tag{6.14.24}$$

The cdf is

$$\text{cdf} \equiv P(< x) \equiv \int_0^x p(x')dx' = 1 - e^{-(x/\beta)^\alpha} \tag{6.14.25}$$

The inverse cdf is

$$x(P) = \beta\,[-\log(1-P)]^{1/\alpha} \tag{6.14.26}$$

For $0 < \alpha < 1$, the distribution has an infinite (but integrable) cusp at $x = 0$ and is monotonically decreasing. The exponential distribution is the case of $\alpha = 1$. When $\alpha > 1$, the distribution is zero at $x = 0$ and has a single maximum at the value $x = \beta\,[(\alpha-1)/\alpha]^{1/\alpha}$.

The mean and variance are given by

$$\begin{aligned} \mu &= \beta\,\Gamma(1+\alpha^{-1}) \\ \sigma^2 &= \beta^2\left\{\Gamma(1+2\alpha^{-1}) - \left[\Gamma(1+\alpha^{-1})\right]^2\right\} \end{aligned} \tag{6.14.27}$$

With correct normalization, equation (6.14.22) becomes

$$\text{hazard} \equiv \frac{p(x)}{P(>x)} = \left(\frac{\alpha}{\beta}\right)\left(\frac{x}{\beta}\right)^{\alpha-1} \tag{6.14.28}$$

6.14.7 Lognormal Distribution

Many processes that live on the positive x-axis are naturally approximated by normal distributions on the "$\log(x)$-axis," that is, for $-\infty < \log(x) < \infty$. A simple, but important, example is the multiplicative random walk, which starts at some positive value x_0, and then generates new values by a recurrence like

$$x_{i+1} = \begin{cases} x_i(1+\epsilon) & \text{with probability } 0.5 \\ x_i/(1+\epsilon) & \text{with probability } 0.5 \end{cases} \tag{6.14.29}$$

Here ϵ is some small, fixed, constant.

These considerations motivate the definition

$$x \sim \text{Lognormal}(\mu,\sigma) \qquad \text{iff} \qquad u \equiv \frac{\log(x)-\mu}{\sigma} \sim \text{N}(0,1) \tag{6.14.30}$$

or the equivalent definition

$$\begin{aligned} &x \sim \text{Lognormal}(\mu,\sigma), \qquad \sigma > 0 \\ &p(x) = \frac{1}{\sqrt{2\pi}\sigma x}\exp\left(-\frac{1}{2}\left[\frac{\log(x)-\mu}{\sigma}\right]^2\right), \qquad x > 0 \end{aligned} \tag{6.14.31}$$

Note the required extra factor of x^{-1} in front of the exponential: The density that is "normal" is $p(\log x)d\log x$.

While μ and σ are the mean and standard deviation in $\log x$ space, they are *not* so in x space. Rather,

$$\begin{aligned} \text{Mean}\{\text{Lognormal}(\mu,\sigma)\} &= e^{\mu+\frac{1}{2}\sigma^2} \\ \text{Var}\{\text{Lognormal}(\mu,\sigma)\} &= e^{2\mu}\,e^{\sigma^2}(e^{\sigma^2}-1) \end{aligned} \tag{6.14.32}$$

The cdf is given by

$$\text{cdf} \equiv P(< x) \equiv \int_0^x p(x')dx' = \frac{1}{2}\text{erfc}\left(-\frac{1}{\sqrt{2}}\left[\frac{\log(x)-\mu}{\sigma}\right]\right) \tag{6.14.33}$$

The inverse to the cdf involves the inverse complementary error function,

$$x(P) = \exp[\mu - \sqrt{2}\sigma\,\text{erfc}^{-1}(2P)] \tag{6.14.34}$$

erf.h

```
struct Lognormaldist : Erf {
Lognormal distribution, derived from the error function Erf.
    Doub mu, sig;
    Lognormaldist(Doub mmu = 0., Doub ssig = 1.) : mu(mmu), sig(ssig) {
        if (sig <= 0.) throw("bad sig in Lognormaldist");
    }
    Doub p(Doub x) {
    Return probability density function.
        if (x < 0.) throw("bad x in Lognormaldist");
        if (x == 0.) return 0.;
        return (0.398942280401432678/(sig*x))*exp(-0.5*SQR((log(x)-mu)/sig));
    }
    Doub cdf(Doub x) {
    Return cumulative distribution function.
        if (x < 0.) throw("bad x in Lognormaldist");
        if (x == 0.) return 0.;
        return 0.5*erfc(-0.707106781186547524*(log(x)-mu)/sig);
    }
    Doub invcdf(Doub p) {
    Return inverse cumulative distribution function.
        if (p <= 0. || p >= 1.) throw("bad p in Lognormaldist");
        return exp(-1.41421356237309505*sig*inverfc(2.*p)+mu);
    }
};
```

Multiplicative random walks like (6.14.29) and lognormal distributions are key ingredients in the economic theory of efficient markets, leading to (among many other results) the celebrated *Black-Scholes formula* for the probability distribution of the price of an investment after some elapsed time τ. A key piece of the Black-Scholes derivation is implicit in equation (6.14.32): If an investment's average return is zero (which may be true in the limit of zero risk), then its price cannot simply be a widening lognormal distribution with fixed μ and increasing σ, for its expected value would then diverge to infinity! The actual Black-Scholes formula thus defines both how σ increases with time (basically as $\tau^{1/2}$) *and* how μ correspondingly decreases with time, so as to keep the overall mean under control. A simplified version of the Black-Scholes formula can be written as

$$S(\tau) \sim S(0) \times \text{Lognormal}\left(r\tau - \tfrac{1}{2}\sigma^2\tau,\ \sigma\sqrt{\tau}\right) \tag{6.14.35}$$

where $S(\tau)$ is the price of a stock at time τ, r is its expected (annualized) rate of return, and σ is now redefined to be the stock's (annualized) volatility. The definition of volatility is that, for small values of τ, the fractional variance of the stock's price is $\sigma^2\tau$. You can check that (6.14.35) has the desired expectation value $E[S(\tau)] = S(0)$, for all τ, if $r = 0$. A good reference is [3].

6.14.8 Chi-Square Distribution

The *chi-square* (or χ^2) distribution has a single parameter $\nu > 0$ that controls both the location and width of its peak. In most applications ν is an integer and is referred to as the *number of degrees of freedom* (see §14.3).

The defining probability density is

$$\begin{aligned} \chi^2 &\sim \text{Chisquare}(\nu), \qquad \nu > 0 \\ p(\chi^2)d\chi^2 &= \frac{1}{2^{\frac{1}{2}\nu}\Gamma(\frac{1}{2}\nu)}(\chi^2)^{\frac{1}{2}\nu-1}\exp\left(-\tfrac{1}{2}\chi^2\right)d\chi^2, \qquad \chi^2 > 0 \end{aligned} \tag{6.14.36}$$

where we have written the differentials $d\chi^2$ merely to emphasize that χ^2, not χ, is to be viewed as the independent variable.

The mean and variance are given by

$$\begin{aligned} \text{Mean}\{\text{Chisquare}(\nu)\} &= \nu \\ \text{Var}\{\text{Chisquare}(\nu)\} &= 2\nu \end{aligned} \tag{6.14.37}$$

When $\nu \geq 2$ there is a single mode at $\chi^2 = \nu - 2$.

The chi-square distribution is actually just a special case of the gamma distribution, below, so its cdf is given by an incomplete gamma function $P(a, x)$,

$$\text{cdf} \equiv P(< \chi^2) \equiv P(\chi^2|\nu) \equiv \int_0^{\chi^2} p(\chi^{2\prime})d\chi^{2\prime} = P\left(\frac{\nu}{2}, \frac{\chi^2}{2}\right) \tag{6.14.38}$$

One frequently also sees the complement of the cdf, which can be calculated either from the incomplete gamma function $P(a, x)$, or from its complement $Q(a, x)$ (often more accurate if P is very close to 1):

$$Q(\chi^2|\nu) \equiv 1 - P(\chi^2|n) = 1 - P\left(\frac{\nu}{2}, \frac{\chi^2}{2}\right) \equiv Q\left(\frac{\nu}{2}, \frac{\chi^2}{2}\right) \tag{6.14.39}$$

The inverse cdf is given in terms of the function that is the inverse of $P(a, x)$ on its second argument, which we here denote $P^{-1}(a, p)$:

$$x(P) = 2P^{-1}\left(\frac{\nu}{2}, P\right) \tag{6.14.40}$$

ncgammabeta.h

```
struct Chisqdist : Gamma {
```

χ^2 distribution, derived from the gamma function `Gamma`.

```
    Doub nu,fac;
    Chisqdist(Doub nnu) : nu(nnu) {
```

Constructor. Initialize with ν.

```
        if (nu <= 0.) throw("bad nu in Chisqdist");
        fac = 0.693147180559945309*(0.5*nu)+gammln(0.5*nu);
    }
    Doub p(Doub x2) {
```

Return probability density function.

```
        if (x2 <= 0.) throw("bad x2 in Chisqdist");
        return exp(-0.5*(x2-(nu-2.)*log(x2))-fac);
    }
    Doub cdf(Doub x2) {
```

```
    Return cumulative distribution function.
        if (x2 < 0.) throw("bad x2 in Chisqdist");
        return gammp(0.5*nu,0.5*x2);
    }
    Doub invcdf(Doub p) {
    Return inverse cumulative distribution function.
        if (p < 0. || p >= 1.) throw("bad p in Chisqdist");
        return 2.*invgammp(p,0.5*nu);
    }
};
```

6.14.9 Gamma Distribution

The *gamma distribution* is defined by

$$\begin{aligned} x &\sim \text{Gamma}(\alpha, \beta), \qquad \alpha > 0,\ \beta > 0 \\ p(x) &= \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}, \qquad x > 0 \end{aligned} \tag{6.14.41}$$

The exponential distribution is the special case with $\alpha = 1$. The chi-square distribution is the special case with $\alpha = \nu/2$ and $\beta = 1/2$.

The mean and variance are given by,

$$\begin{aligned} \text{Mean}\{\text{Gamma}(\alpha, \beta)\} &= \alpha/\beta \\ \text{Var}\{\text{Gamma}(\alpha, \beta)\} &= \alpha/\beta^2 \end{aligned} \tag{6.14.42}$$

When $\alpha \geq 1$ there is a single mode at $x = (\alpha - 1)/\beta$.

Evidently, the cdf is the incomplete gamma function

$$\text{cdf} \equiv P(< x) \equiv \int_0^x p(x')dx' = P(\alpha, \beta x) \tag{6.14.43}$$

while the inverse cdf is given in terms of the inverse of $P(a, x)$ on its second argument by

$$x(P) = \frac{1}{\beta} P^{-1}(\alpha, P) \tag{6.14.44}$$

incgammabeta.h

```
struct Gammadist : Gamma {
Gamma distribution, derived from the gamma function Gamma.
    Doub alph, bet, fac;
    Gammadist(Doub aalph, Doub bbet = 1.) : alph(aalph), bet(bbet) {
    Constructor. Initialize with α and β.
        if (alph <= 0. || bet <= 0.) throw("bad alph,bet in Gammadist");
        fac = alph*log(bet)-gammln(alph);
    }
    Doub p(Doub x) {
    Return probability density function.
        if (x <= 0.) throw("bad x in Gammadist");
        return exp(-bet*x+(alph-1.)*log(x)+fac);
    }
    Doub cdf(Doub x) {
    Return cumulative distribution function.
        if (x < 0.) throw("bad x in Gammadist");
        return gammp(alph,bet*x);
    }
```

```
    Doub invcdf(Doub p) {
    Return inverse cumulative distribution function.
        if (p < 0. || p >= 1.) throw("bad p in Gammadist");
        return invgammp(p,alph)/bet;
    }
};
```

6.14.10 F-Distribution

The *F-distribution* is parameterized by two positive values ν_1 and ν_2, usually (but not always) integers.

The defining probability density is

$$\begin{aligned} F &\sim \mathrm{F}(\nu_1,\nu_2), \qquad \nu_1>0,\ \nu_2>0 \\ p(F) &= \frac{\nu_1^{\frac{1}{2}\nu_1}\nu_2^{\frac{1}{2}\nu_2}}{B(\frac{1}{2}\nu_1,\frac{1}{2}\nu_2)}\frac{F^{\frac{1}{2}\nu_1-1}}{(\nu_2+\nu_1 F)^{(\nu_1+\nu_2)/2}}, \qquad F>0 \end{aligned} \tag{6.14.45}$$

where $B(a,b)$ denotes the beta function. The mean and variance are given by

$$\begin{aligned} \mathrm{Mean}\{\mathrm{F}(\nu_1,\nu_2)\} &= \frac{\nu_2}{\nu_2-2}, && \nu_2>2 \\ \mathrm{Var}\{\mathrm{F}(\nu_1,\nu_2)\} &= \frac{2\nu_2^2(\nu_1+\nu_2-2)}{\nu_1(\nu_2-2)^2(\nu_2-4)}, && \nu_2>4 \end{aligned} \tag{6.14.46}$$

When $\nu_1 \geq 2$ there is a single mode at

$$F = \frac{\nu_2(\nu_1-2)}{\nu_1(\nu_2+2)} \tag{6.14.47}$$

.

For fixed ν_1, if $\nu_2 \to \infty$, the F-distribution becomes a chi-square distribution, namely

$$\lim_{\nu_2\to\infty} \mathrm{F}(\nu_1,\nu_2) \cong \frac{1}{\nu_1}\mathrm{Chisquare}(\nu_1) \tag{6.14.48}$$

where "$\cong$" means "are identical distributions."

The F-distribution's cdf is given in terms of the incomplete beta function $I_x(a,b)$ by

$$\mathrm{cdf} \equiv P(<x) \equiv \int_0^x p(x')dx' = I_{\nu_1 F/(\nu_2+\nu_1 F)}\left(\tfrac{1}{2}\nu_1,\tfrac{1}{2}\nu_2\right) \tag{6.14.49}$$

while the inverse cdf is given in terms of the inverse of $I_x(a,b)$ on its subscript argument by

$$\begin{aligned} u &\equiv I_p^{-1}\left(\tfrac{1}{2}\nu_1,\tfrac{1}{2}\nu_2\right) \\ x(P) &= \frac{\nu_2 u}{\nu_1(1-u)} \end{aligned} \tag{6.14.50}$$

A frequent use of the F-distribution is to test whether two observed samples have the same variance.

incgammabeta.h

```
struct Fdist : Beta {
```
F distribution, derived from the beta function `Beta`.
```
    Doub nu1,nu2;
    Doub fac;
    Fdist(Doub nnu1, Doub nnu2) : nu1(nnu1), nu2(nnu2) {
```
Constructor. Initialize with ν_1 and ν_2.
```
        if (nu1 <= 0. || nu2 <= 0.) throw("bad nu1,nu2 in Fdist");
        fac = 0.5*(nu1*log(nu1)+nu2*log(nu2))+gammln(0.5*(nu1+nu2))
            -gammln(0.5*nu1)-gammln(0.5*nu2);
    }
    Doub p(Doub f) {
```
Return probability density function.
```
        if (f <= 0.) throw("bad f in Fdist");
        return exp((0.5*nu1-1.)*log(f)-0.5*(nu1+nu2)*log(nu2+nu1*f)+fac);
    }
    Doub cdf(Doub f) {
```
Return cumulative distribution function.
```
        if (f < 0.) throw("bad f in Fdist");
        return betai(0.5*nu1,0.5*nu2,nu1*f/(nu2+nu1*f));
    }
    Doub invcdf(Doub p) {
```
Return inverse cumulative distribution function.
```
        if (p <= 0. || p >= 1.) throw("bad p in Fdist");
        Doub x = invbetai(p,0.5*nu1,0.5*nu2);
        return nu2*x/(nu1*(1.-x));
    }
};
```

6.14.11 Beta Distribution

The *beta distribution* is defined on the unit interval $0 < x < 1$ by

$$\begin{aligned} x &\sim \text{Beta}(\alpha, \beta), \qquad \alpha > 0,\ \beta > 0 \\ p(x) &= \frac{1}{B(\alpha, \beta)} x^{\alpha-1}(1-x)^{\beta-1}, \qquad 0 < x < 1 \end{aligned} \tag{6.14.51}$$

The mean and variance are given by

$$\begin{aligned} \text{Mean}\{\text{Beta}(\alpha, \beta)\} &= \frac{\alpha}{\alpha+\beta} \\ \text{Var}\{\text{Beta}(\alpha, \beta)\} &= \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)} \end{aligned} \tag{6.14.52}$$

When $\alpha > 1$ and $\beta > 1$, there is a single mode at $(\alpha-1)/(\alpha+\beta-2)$. When $\alpha < 1$ and $\beta < 1$, the distribution function is "U-shaped" with a minimum at this same value. In other cases there is neither a maximum nor a minimum.

In the limit that β becomes large as α is held fixed, all the action in the beta distribution shifts toward $x = 0$, and the density function takes the shape of a gamma distribution. More precisely,

$$\lim_{\beta\to\infty} \beta\, \text{Beta}(\alpha, \beta) \cong \text{Gamma}(\alpha, 1) \tag{6.14.53}$$

The cdf is the incomplete beta function

$$\text{cdf} \equiv P(< x) \equiv \int_0^x p(x')dx' = I_x(\alpha, \beta) \tag{6.14.54}$$